



NUST CHIP DESIGN CENTRE

RISC-V Architecture

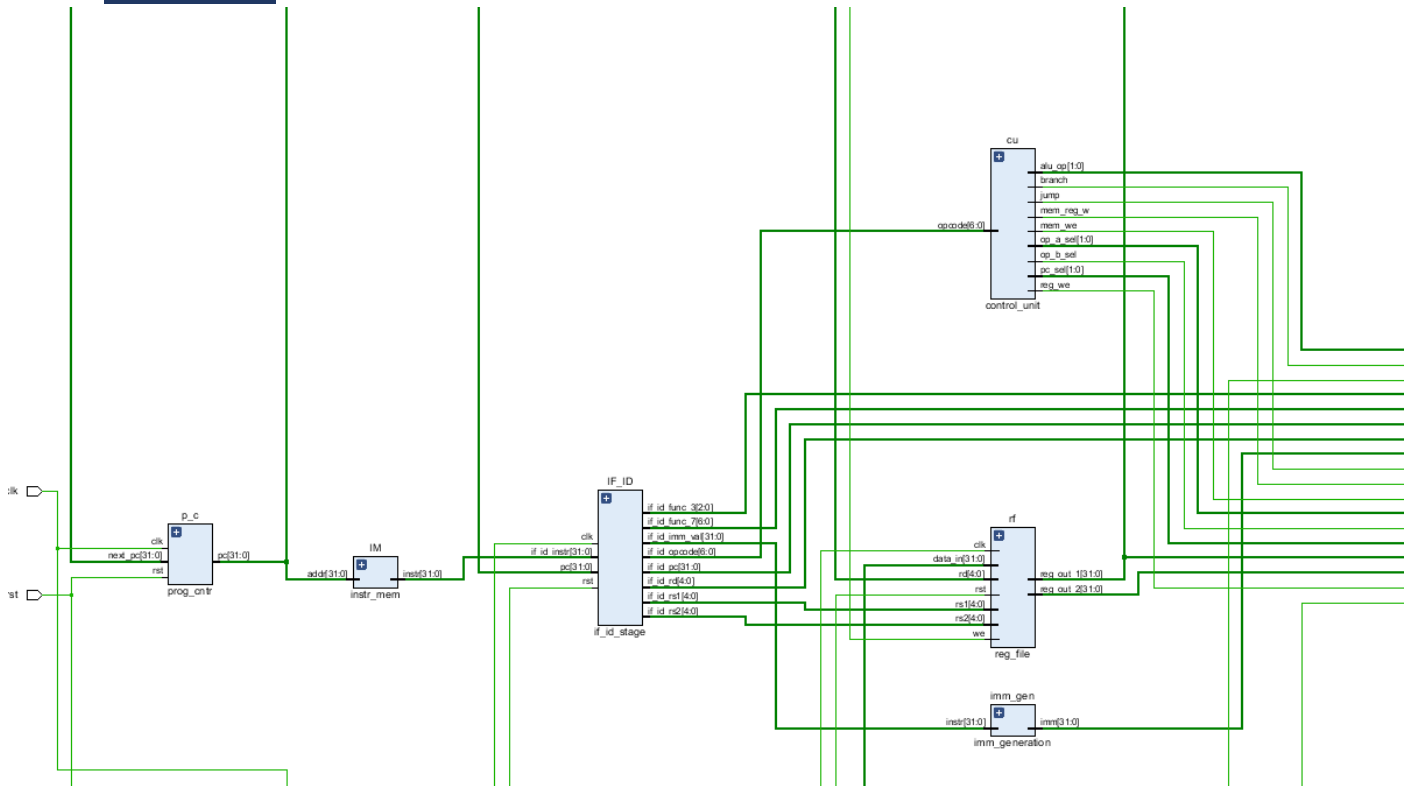
Lab Manual # 23 **Handling Control Hazards**

<u>Name</u>	<i><u>Muddassir Ali Siddiqui</u></i>
<u>Instructor</u>	<i><u>Sir Imran & Sir Umer</u></i>
<u>Date</u>	<i><u>5th Sep 2025</u></i>

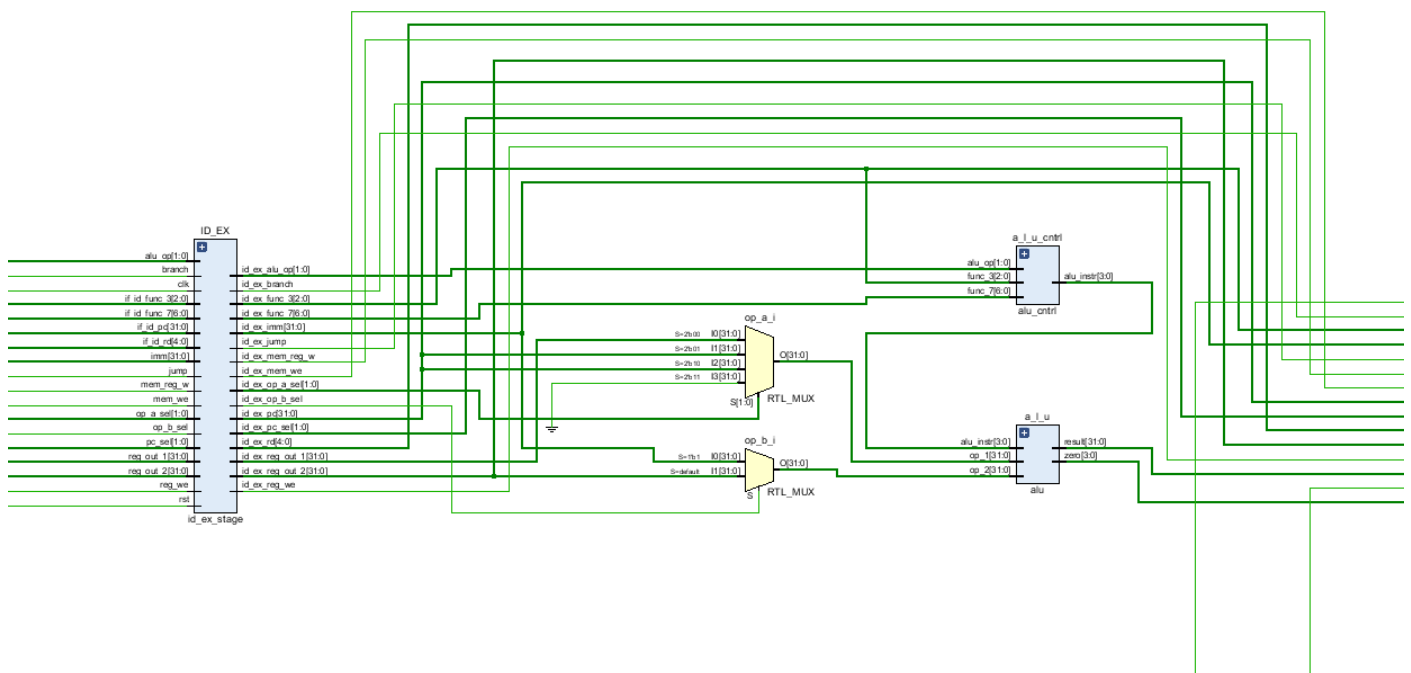
1. In-Lab Tasks: (Write your lab task & screenshots here)

i. Schematics:

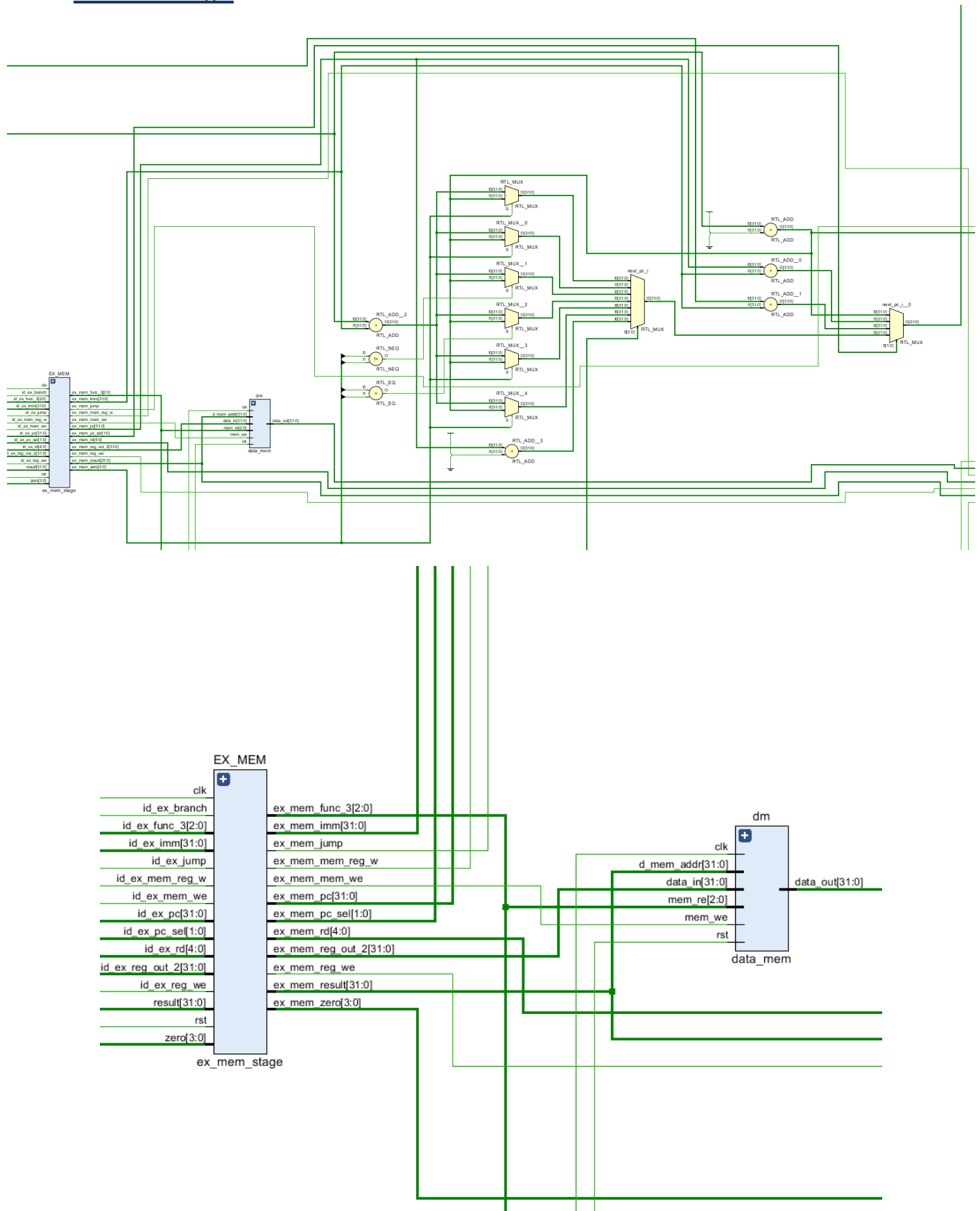
a. IF-ID Stage:



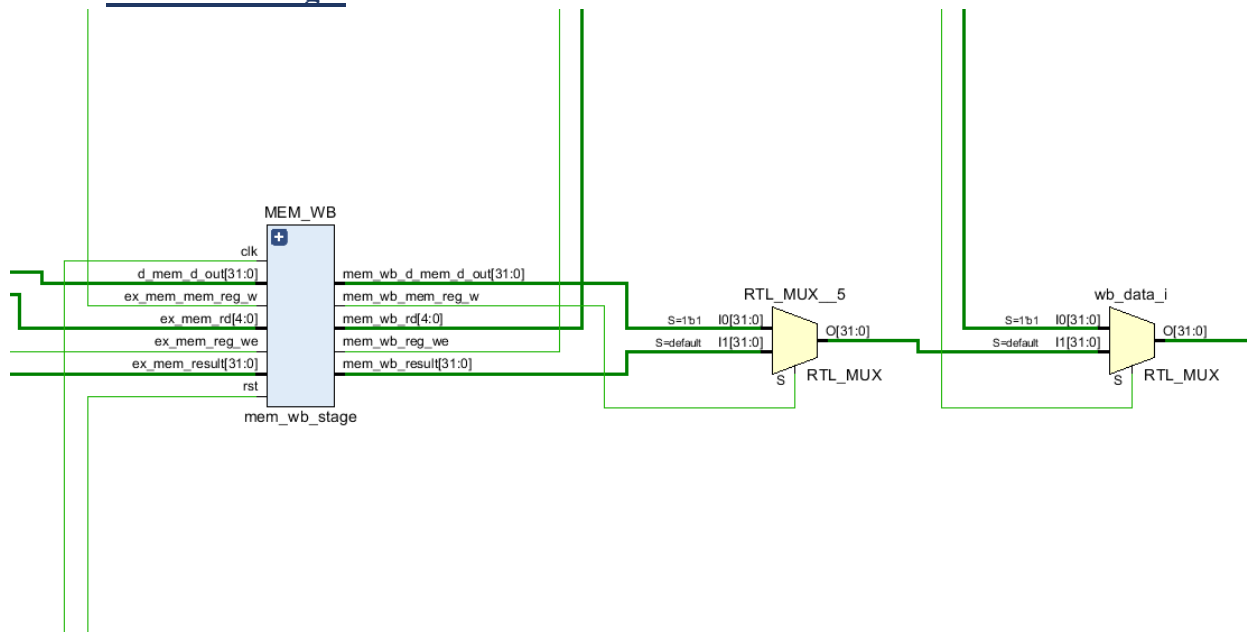
b. ID-EX Stage:



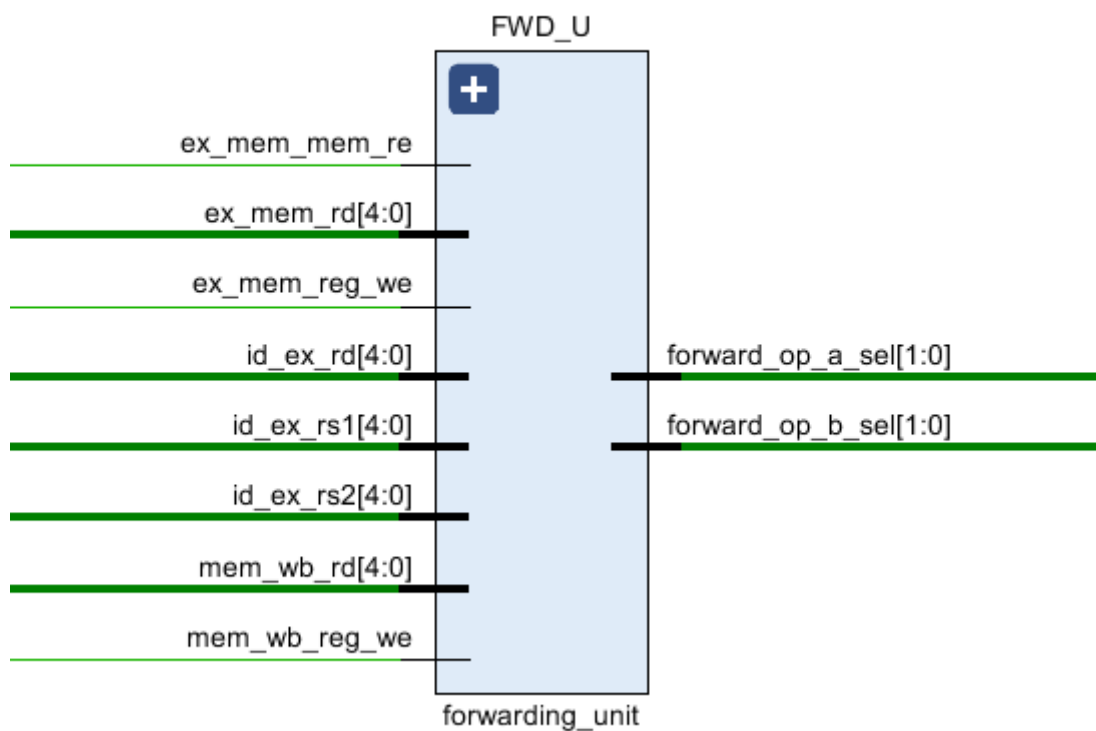
c. EX-MEM Stage:

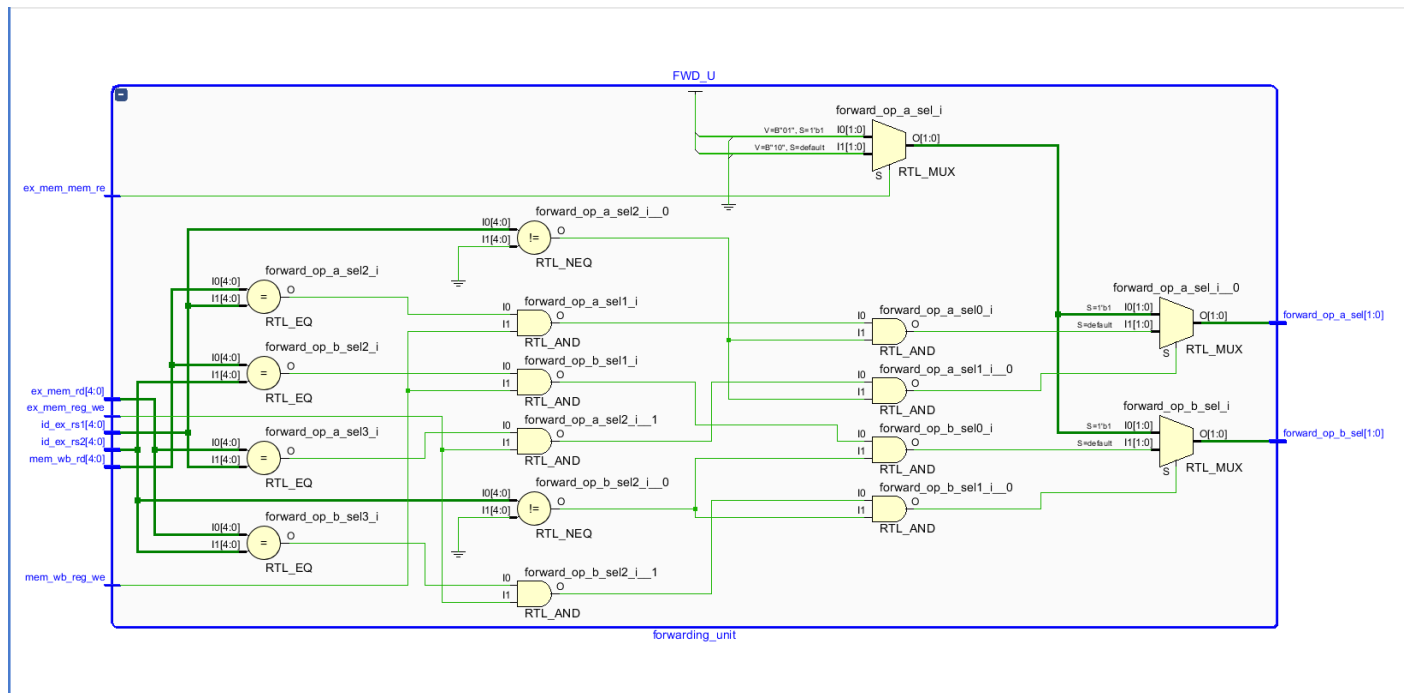


d. MEM-WB Stage:

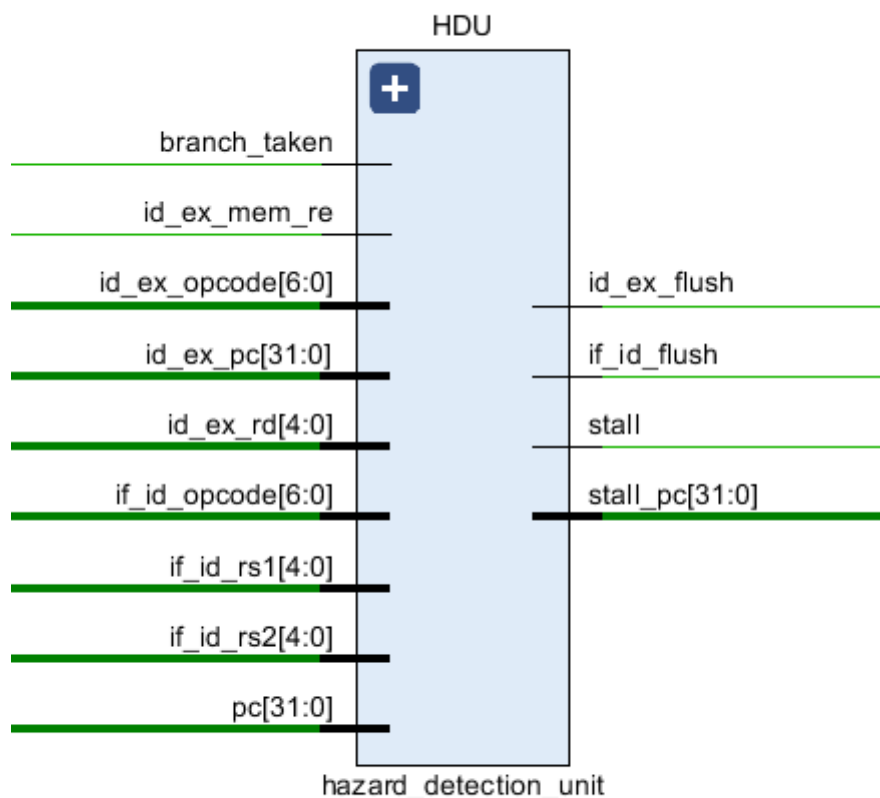


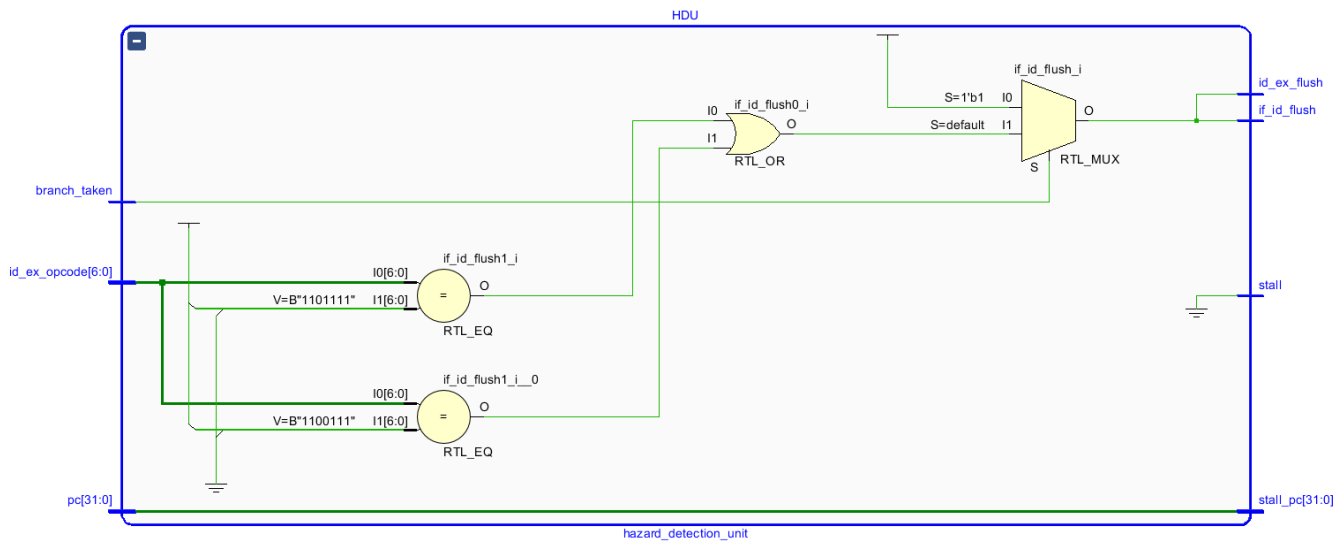
e. Forwarding Unit:



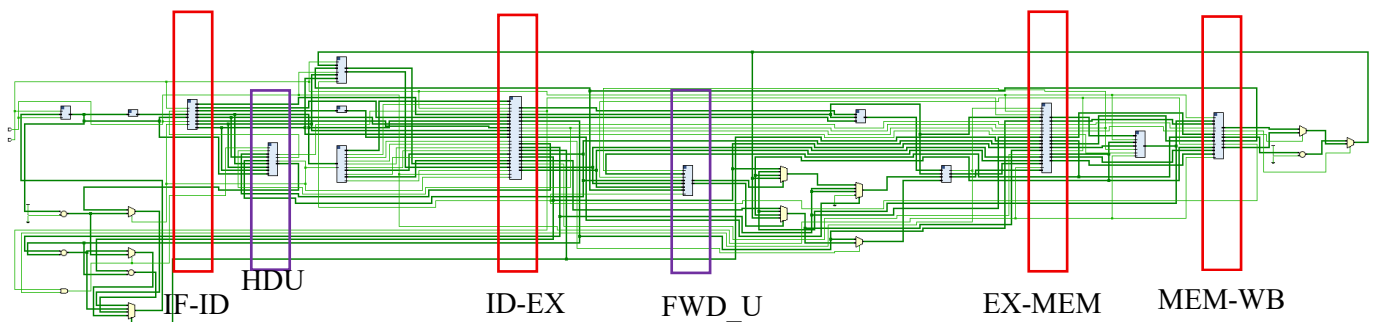


f. Hazard Detection Unit:





g. TOP-5-Stage-Pipeline with handling data Hazards:



ii. Simulation:

Venus Code:

```
addi x1, x0, 0x10
add x2, x1, x1
add x3, x1, x2
add x4, x1, x3
add x5, x1, x4
add x6, x1, x5
```

```
addi x1, x0, 0x100
sw x1, 0(x1)
lw x2, 0(x1)
addi x3, x2, 0x100    # x3 = 100
add x7, x3, x3        # x7 = 7
```

Screenshots:

ra (x1)	0x00000010	register_file[0:31][31:0]	00000000,0000
sp (x2)	0x00000020	> [0][31:0]	00000000
gp (x3)	0x00000030	> [1][31:0]	00000010
tp (x4)	0x00000040	> [2][31:0]	00000020
t0 (x5)	0x00000050	> [3][31:0]	00000030
t1 (x6)	0x00000060	> [4][31:0]	00000040
		> [5][31:0]	00000050
		> [6][31:0]	00000060
		> [7][31:0]	00000000

zero	0x00000000	data_mem[0:1023][31:0]	00000000,0000
ra (x1)	0x00000100	> [64][31:0]	00000100
sp (x2)	0x00000100	> [128][31:0]	00000000
gp (x3)	0x00000200	register_file[0:31][31:0]	00000000,0000
tp (x4)	0x00000000	> [0][31:0]	00000000
t0 (x5)	0x00000000	> [1][31:0]	00000100
t1 (x6)	0x00000000	> [2][31:0]	00000100
t2 (x7)	0x00000400	> [3][31:0]	00000200
		> [4][31:0]	00000000
		> [5][31:0]	00000000
		> [6][31:0]	00000000
		> [7][31:0]	00000400
		> [8][31:0]	00000000

These are results comparison. You will observe that the results are incredibly true.

iii. RISC V Tester:

h. Given Venus Code:

```
# Test the RISC-V processor:

# add, sub, and, or, slt, addi, lw, sw, beq, jal

# If successful, it should write the value 25 to address 100

#

# RISC-V Assembly      Description      Address  Machine Code
main: addi x2, x0, 5    # x2 = 5          0        00500113
      addi x3, x0, 12   # x3 = 12          4        00c00193
      addi x7, x3, -9   # x7 = (12 - 9) = 3    8        ff718393
      or  x4, x7, x2    # x4 = (3 OR 5) = 7    C        0023e233
      and x5, x3, x4    # x5 = (12 AND 7) = 4   10       0041f2b3
      add x5, x5, x4    # x5 = 4 + 7 = 11     14       004282b3
      beq x5, x7, end   # shouldn't be taken  18       02728663
      slt x4, x3, x4    # x4 = (12 < 7) = 0   1C       40720333
      beq x4, x0, around # should be taken   20       00020663
      addi x5, x0, 0    # shouldn't execute   24       00000293
around: slt x4, x7, x2  # x6 = (3 < 5) = 1    28       004323b3
      add x7, x4, x5    # x6 = (1 + 11) = 12   2C       0062a2b3
      sub x7, x7, x2    # x7 = (12 - 5) = 7    30       0023a3b3
      sw  x7, 84(x3)    # [96] = 7           34       0471aa23
      lw  x2, 96(x0)    # x2 = [96] = 7       38       06012083
      add x9, x2, x5    # x9 = (7 + 11) = 18   3C       005104b3
      jal x3, end      # jump to end, x3 = 0x44 40       008000ef
      addi x2, x0, 1    # shouldn't execute   44       00100113
end: add x2, x2, x9     # x2 = (7 + 18) = 25   48       00910133
      sw  x2, 0x20(x3)  # [100] = 25         4C       0621a023
done: beq x2, x2, done  # infinite loop      50       00210063
```



```

1 # Test the RISC-V processor:
2 #   add, sub, and, or, slt, addi, lw, sw, beq, jal
3 # If successful, it should write the value 25 to address 100
4 #
5 #   RISC-V Assembly      Description      Address  Machine Code
6 main: addi x2, x0, 5      # x2 = 5      0        00500113
7      addi x3, x0, 12      # x3 = 12      4        00c00193
8      addi x7, x3, -9      # x7 = (12 - 9) = 3    8        f f718393
9      or   x4, x7, x2      # x4 = (3 OR 5) = 7    C        0023e233
10     and  x5, x3, x4      # x5 = (12 AND 7) = 4    10       0041f2b3
11     add  x5, x5, x4      # x5 = 4 + 7 = 11    14       004282b3
12     beq  x5, x7, end     # shouldn't be taken    18       02728663
13     slt  x4, x3, x4      # x4 = (12 - 7) = 0    1C       40720333
14     beq  x4, x0, around  # should be taken    20       00020663
15     addi x5, x0, 0       # shouldn't execute    24       00000293
16 around: slt x4, x7, x2  # x6 = (3 < 5) = 1    28       004323b3
17     add  x7, x4, x5      # x6 = (1 < 11) = 12    2C       0062a2b3
18     sub  x7, x7, x2      # x7 = (12 + 5) = 7    30       0023a3b3
19     sw   x7, 84(x3)      # [96] = 7          34       0471aa23
20     lw   x2, 96(x0)      # x2 = [96] = 7      38       06012083
21     add  x9, x2, x5      # x9 = (7 + 11) = 18  3C       005104b3
22     jal  x3, end         # jump to end, x3 = 0x44 40       008000ef
23     addi x2, x0, 1       # shouldn't execute    44       00100113
24 end:  add  x2, x2, x9     # x2 = (7 + 18) = 25  48       00910133
25     sw   x2, 0x20(x3)    # [100] = 25        4C       0621a023
26 done: beq  x2, x2, done  # infinite loop      50       00210063
27

```

i. Instruction Memory:

> pc[31:0]	00000050	00000000	00000004	00000008	0000000c	00000010	00000014	00000018	0000001c	00000020	00000028	0000002c	00000030	00000034
> next_pc[31:0]	00000050	00000004	00000008	0000000c	00000010	00000014	00000018	0000001c	00000020	00000028	0000002c	00000030	00000034	00000038
> instr[31:0]	00500113	00c00193	ff718393	0023e233	0041f2b3	004282b3	02728663	0041a233	00020463	0023a233	005203b3	402383b3	0471aa23	
> pc[31:0]	00000050	00000038	0000003c	00000040	00000048	0000004c				00000050				
> next_pc[31:0]	00000050	0000003c	00000040	00000048	0000004c				00000050					
> instr[31:0]	0471aa23	06002103	005104b3	008001ef	00910133	0221a023				00210063				

j. After 5 iterations:

sp (x2)	0x00000005	register_file[31:0]	00000000,00000000	00000000,00000000
gp (x3)	0x0000000C	> [0][31:0]	00000000	
tp (x4)	0x00000007	> [1][31:0]	00000000	
t0 (x5)	0x00000004	> [2][31:0]	00000005	00000005
t1 (x6)	0x00000000	> [3][31:0]	0000000c	0000000c
t2 (x7)	0x00000003	> [4][31:0]	00000007	00000007
		> [5][31:0]	00000004	00000004
		> [6][31:0]	00000000	00000000
		> [7][31:0]	00000003	00000003
		> [8][31:0]	00000000	00000000
		> [9][31:0]	00000000	00000000
		> [10][31:0]	00000000	00000000
		> [11][31:0]	00000000	00000000
		> [12][31:0]	00000000	00000000

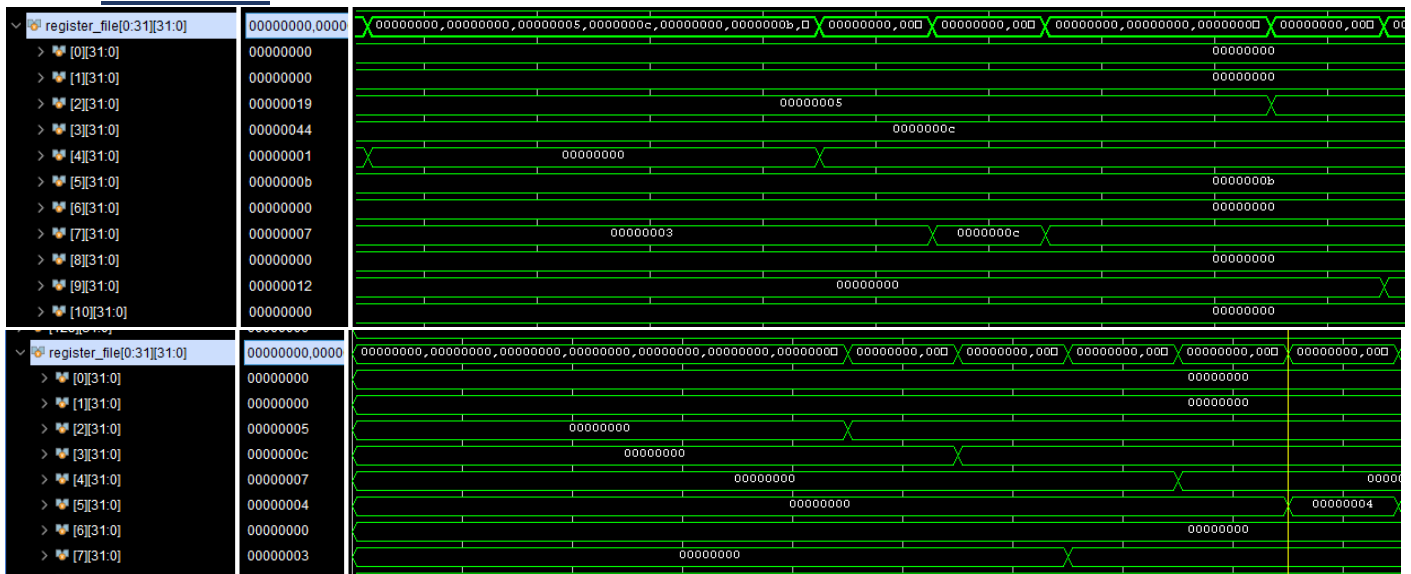
k. After 12 iterations:

sp (x2)	0x00000005	register_file[31:0]	00000000,00000000	00000000,00000000
gp (x3)	0x0000000C	> [0][31:0]	00000000	
tp (x4)	0x00000001	> [1][31:0]	00000000	
t0 (x5)	0x0000000B	> [2][31:0]	00000005	00000005
t1 (x6)	0x00000000	> [3][31:0]	0000000c	0000000c
t2 (x7)	0x0000000C	> [4][31:0]	00000001	00000001
		> [5][31:0]	0000000b	0000000b
		> [6][31:0]	00000000	00000000
		> [7][31:0]	0000000c	0000000c
		> [8][31:0]	00000000	00000000
		> [9][31:0]	00000000	00000000
		> [10][31:0]	00000000	00000000

l. After 18 iterations:

sp (x2)	0x00000007	register_file[31:0]	00000000,00000000	00000000,00000000
gp (x3)	0x00000044	> [0][31:0]	00000000	
tp (x4)	0x00000001	> [1][31:0]	00000000	
t0 (x5)	0x0000000B	> [2][31:0]	00000007	00000007
t1 (x6)	0x00000000	> [3][31:0]	00000044	00000044
t2 (x7)	0x00000007	> [4][31:0]	00000001	00000001
s0 (x8)	0x00000000	> [5][31:0]	0000000b	0000000b
s1 (x9)	0x00000012	> [6][31:0]	00000000	00000000
		> [7][31:0]	00000007	00000007
		> [8][31:0]	00000000	00000000
		> [9][31:0]	00000012	00000012
		> [10][31:0]	00000000	00000000

m. Final result:



n. Branch Hazards:



Here the branch is taken so I flush out the first 2 registers if_id and id_ex, but the jump address is also the second address here that's why the long cycle of same pc = 28.

> if_id_opcode[6:0]	00	> ex_mem_result[31:0]	00000000
> if_id_rs1[4:0]	00	> ex_mem_rs1[4:0]	00
> if_id_rs2[4:0]	00	> ex_mem_rs2[4:0]	00
> if_id_rd[4:0]	00	> ex_mem_rd[4:0]	00
> if_id_func_3[2:0]	0	> ex_mem_reg_out_1[31:0]	00000000
> if_id_func_7[6:0]	00	> ex_mem_reg_out_2[31:0]	00000000
> if_id_instr[31:0]	00000000	🔧 ex_mem_reg_we	0
> if_id_pc[31:0]	00000000	🔧 ex_mem_mem_we	0
> id_ex_reg_out_1[31:0]	00000000	🔧 ex_mem_mem_re	0
> id_ex_reg_out_2[31:0]	00000000	🔧 ex_mem_mem_reg_w	0
> id_ex_rs1[4:0]	00	> ex_mem_pc_sel[1:0]	0
> id_ex_rs2[4:0]	00	🔧 ex_mem_jump	0
> id_ex_rd[4:0]	00	🔧 ex_mem_branch	0
> id_ex_imm[31:0]	00000000	> ex_mem_func_3[2:0]	0
🔧 id_ex_reg_we	0	> ex_mem_pc[31:0]	00000000
🔧 id_ex_mem_we	0	> ex_mem_imm[31:0]	00000000
🔧 id_ex_mem_re	0	🔧 mem_wb_reg_we	0
🔧 id_ex_mem_reg_w	0	🔧 mem_wb_mem_reg_w	0
> id_ex_alu_op[1:0]	0	🔧 mem_wb_jump	0
🔧 id_ex_op_b_sel	0	> mem_wb_rs1[4:0]	00
> id_ex_op_a_sel[1:0]	0	> mem_wb_rs2[4:0]	00
> id_ex_pc_sel[1:0]	0	> mem_wb_rd[4:0]	00
🔧 id_ex_jump	0	> mem_wb_reg_out_1[31:0]	00000000
🔧 id_ex_branch	0	> mem_wb_result[31:0]	00000000
> id_ex_func_3[2:0]	0	> mem_wb_d_mem_d_out[31:0]	00000000
> id_ex_func_7[6:0]	00	> mem_wb_pc[31:0]	00000000
> id_ex_pc[31:0]	00000000	> mem_wb_imm[31:0]	00000000

This is the output of 2 cycles which flush the registers.

2. Critical Analysis: (Write you critical analysis / conclusion here)

In this lab, I successfully implemented the complete **5-stage pipelined RISC-V processor** (IF, ID, EX, MEM, WB). By integrating hazard detection, forwarding, and stalling mechanisms, the design now supports smooth instruction execution with minimal performance penalties. The pipeline improves throughput by allowing multiple instructions to overlap in execution, while control logic ensures correctness even in the presence of hazards. This implementation demonstrates the practical balance between efficiency and reliability in modern processor architectures.